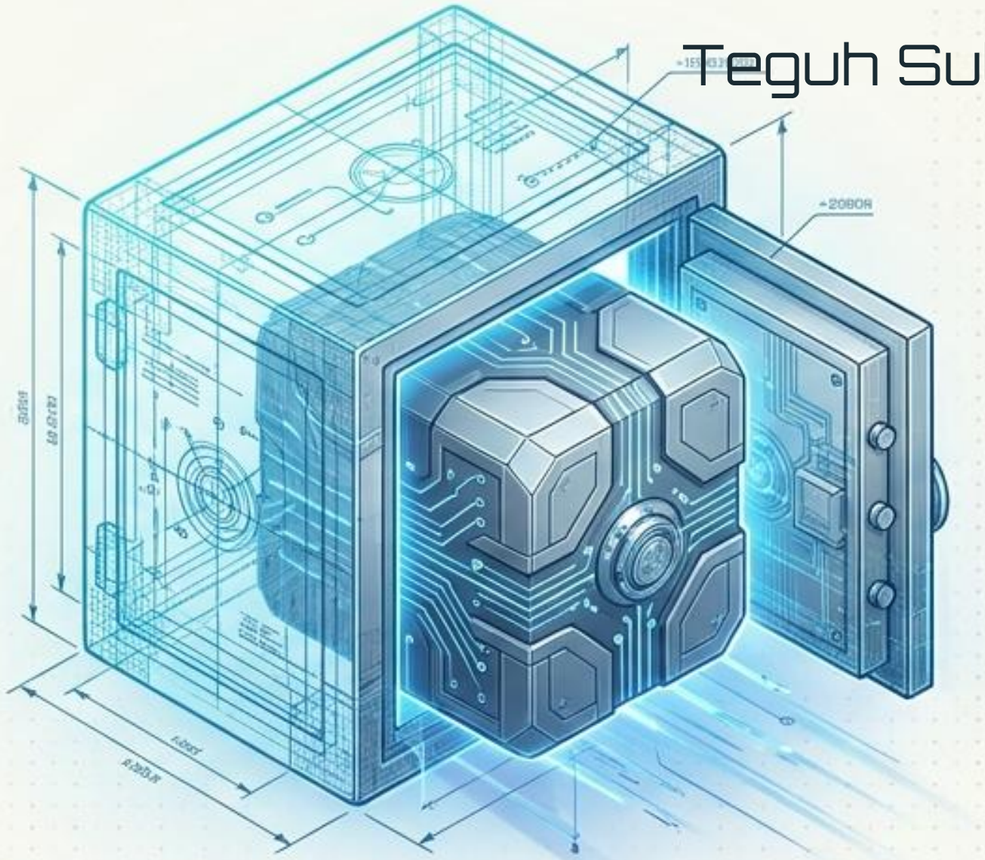
Teguh Sutanto, M.Kom., MCP.



PBO Pert 4:

Konsep OOP & Encapsulation

Membangun Arsitektur Keamanan Objek dalam Java

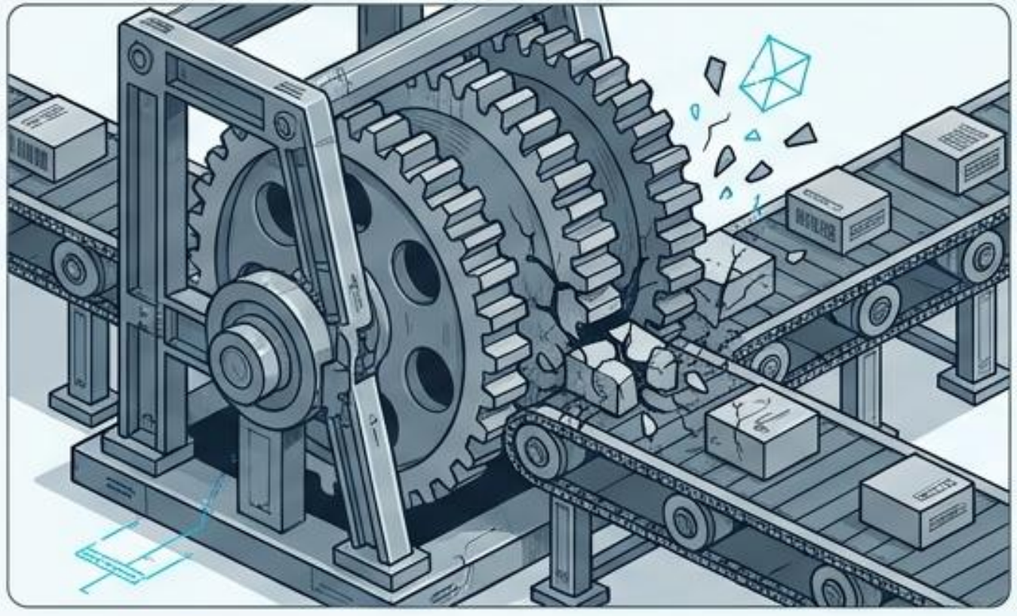
Tim Arsitektur Perangkat Lunak

Dokumen Referensi Teknis

Gemini Notebook

Pergeseran Paradigma: Mengapa OOP Muncul?

Desain Prosedural



Algorithms > Data

Fokus pada urutan instruksi dan prosedur. Data diperlakukan pasif dan terpisah dari fungsi utama.

Desain Berorientasi Objek

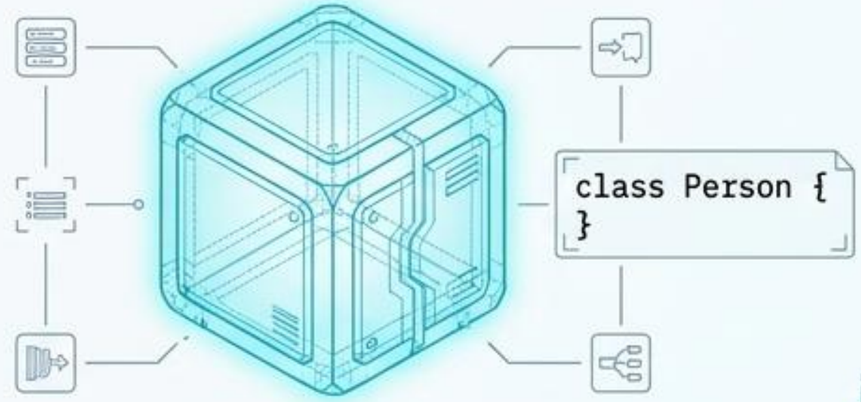


Data > Algorithms

Fokus utama pada objek (data) dan antarmukanya. Fungsi melayani data, bukan sebaliknya.

Gemini Notebook

Fondasi Absolut: Class vs. Object



Class (Cetakan)

Deklarasi atribut dan metode. Sebuah blueprint logis yang belum memakan memori fisik.



Object (Instansiasi)

Entitas fisik di dalam heap memory. Diciptakan dari Class menggunakan keyword 'new'. Setiap objek memiliki eksistensinya sendiri secara independen.

Gemini Notebook

Anatomi Fundamental Sebuah Objek





State (Kondisi)

Kumpulan nilai dari atribut pada suatu waktu tertentu (misal: name, age). State berubah saat metode dipanggil.



Behavior (Perilaku)

Apa yang bisa dilakukan objek tersebut. Didefinisikan oleh metode (misal: showDetails()).

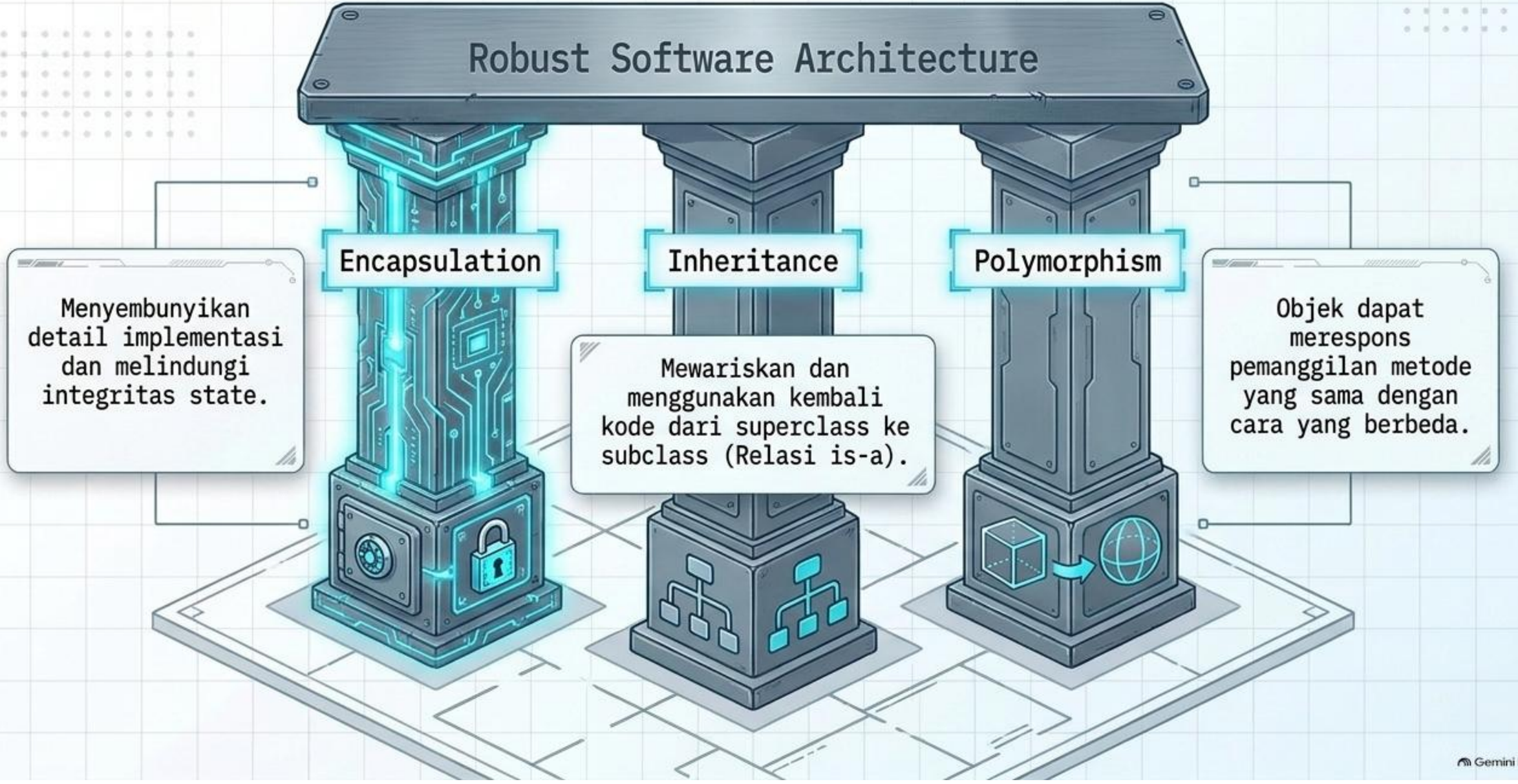


Identity (Identitas)

Referensi unik di dalam memori. Membedakan satu objek dengan objek lain meskipun memiliki state yang sama persis.

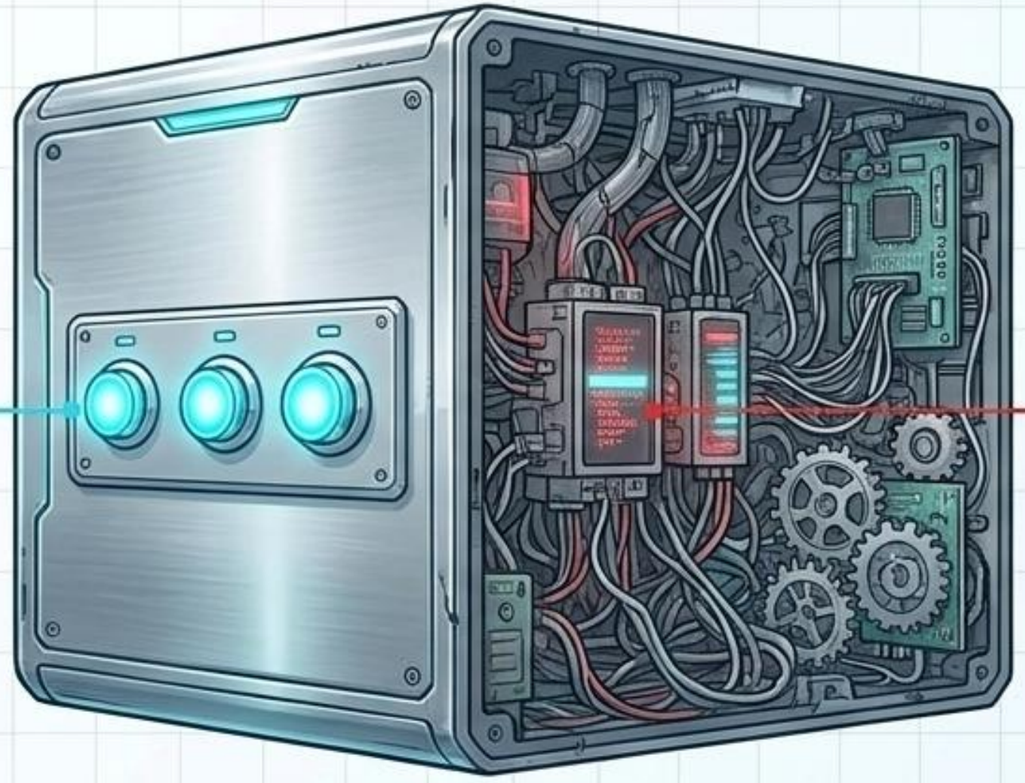
Gemini Notebook

Tiga Pilar Utama Pemrograman Berorientasi Objek



Gemini Notebook

Membedah Encapsulation: Konsep Black Box



Information Hiding

Enkapsulasi membungkus data dan perilaku dalam satu paket. Kode eksternal tidak perlu—dan tidak diizinkan—tahu BAGAIMANA data disimpan atau diproses di dalam kotak hitam tersebut. Interaksi hanya terjadi melalui tombol panel yang disediakan.

Gemini Notebook

Masalah Utama: Mengapa Akses Langsung Berbahaya?

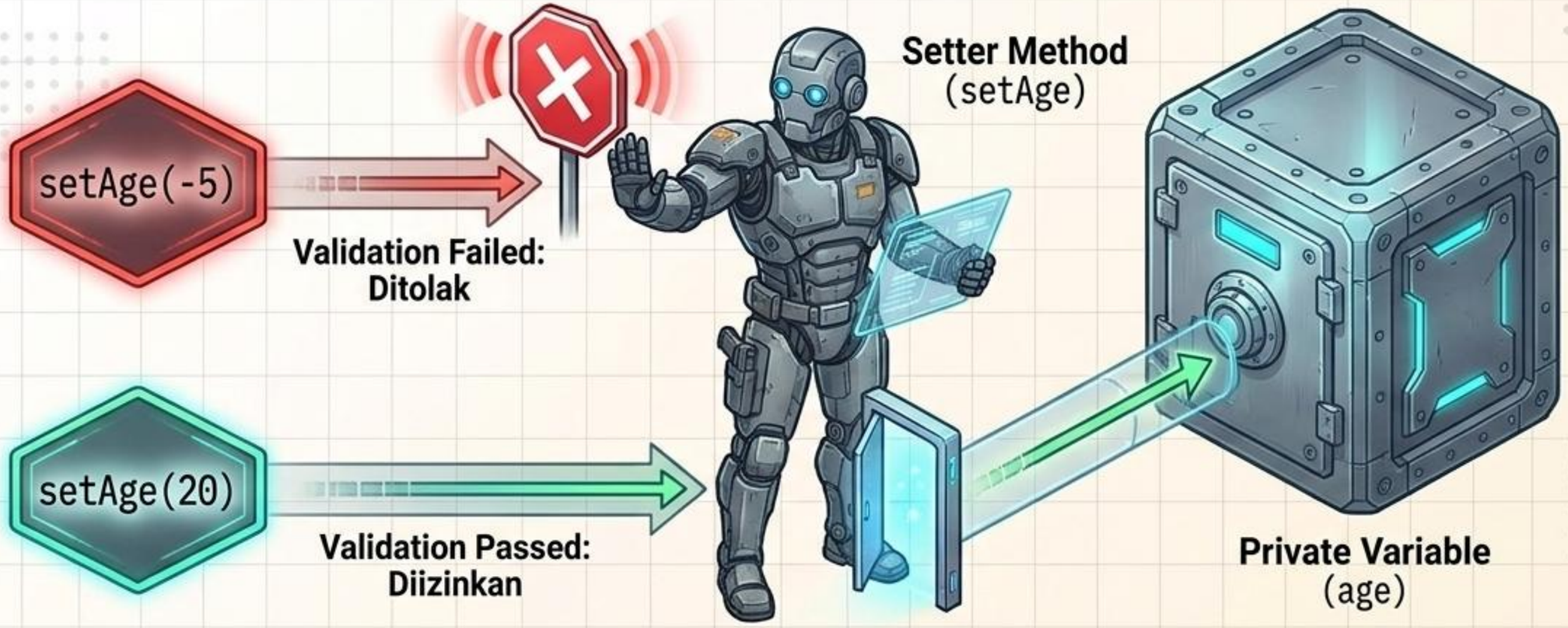


```
1
2
3  Person p2 = new Person();
4  p2.age = -5; // Fatal Logic Error
```

Dalam dunia nyata, umur seseorang tidak mungkin bernilai negatif. Jika atribut dibiarkan 'public', tidak ada mekanisme keamanan yang mencegah kode eksternal memasukkan data yang tidak valid. Hal ini menyebabkan **fragile code** dan **illegal state**.

Gemini Notebook

Solusi: Model Gatekeeper (Penjaga Gerbang)

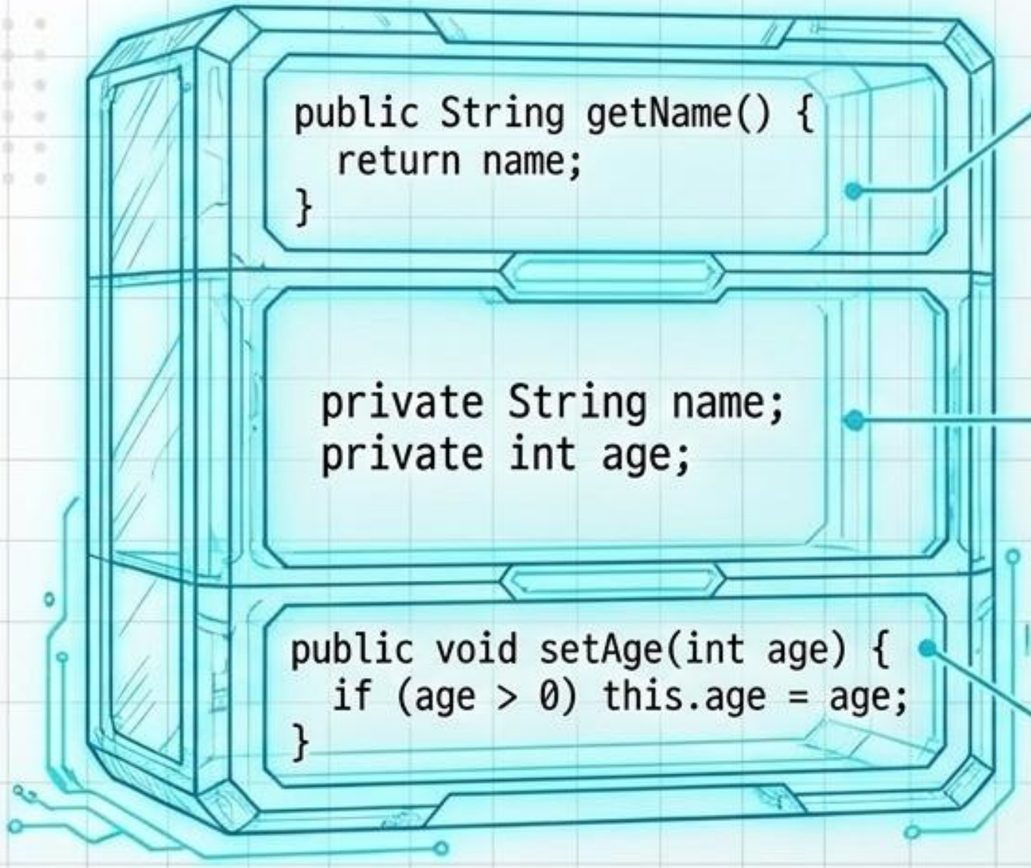


Solusi: Private Fields & Public Methods

Atribut dikunci menjadi 'private'. Interaksi hanya diizinkan melalui jalur resmi (getters dan setters), memungkinkan objek untuk memvalidasi setiap perubahan sebelum data ditulis.

Gemini Notebook

Anatomi Java Bean (POJO)



Akses Baca Terkontrol.
Memberikan salinan data ke pemanggil.

Terisolasi (Data Tertutup).
Variabel disembunyikan dari luar kelas.

Akses Tulis & Validasi.
Mengubah data dengan aturan bisnis.

Konvensi standar di Java: Semua atribut wajib private, diakses murni melalui metode getter dan setter publik.

Gemini Notebook

Matriks Izin Akses (Security Clearance)

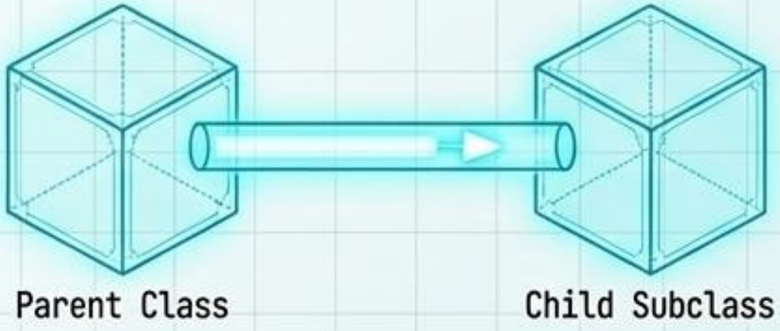
Modifier	Kelas yang Sama	Package yang Sama	Subclass (Beda Package)	Global (Di Mana Saja)
private				
default				
protected				
public				

Tabel Matriks Hak Akses Java. Referensi inti untuk membatasi visibilitas objek.

Gemini Notebook

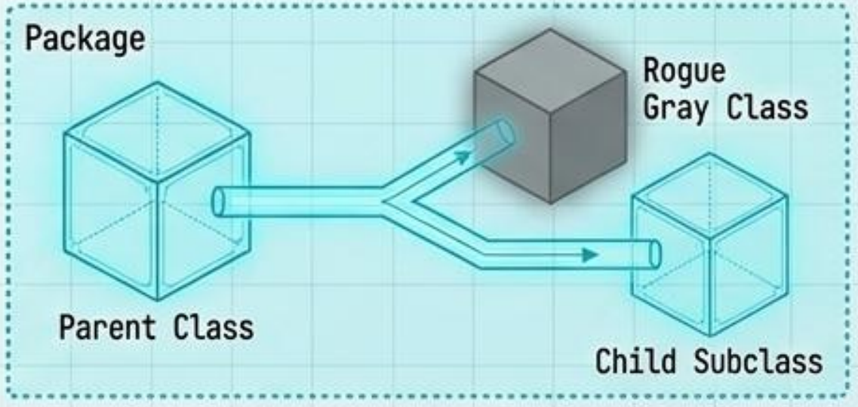
Mengapa Menghindari Akses ‘Protected’?

Mitos



'protected' bagus karena hanya subclass yang bisa mengaksesnya.

Realita (Kebocoran Enkapsulasi)



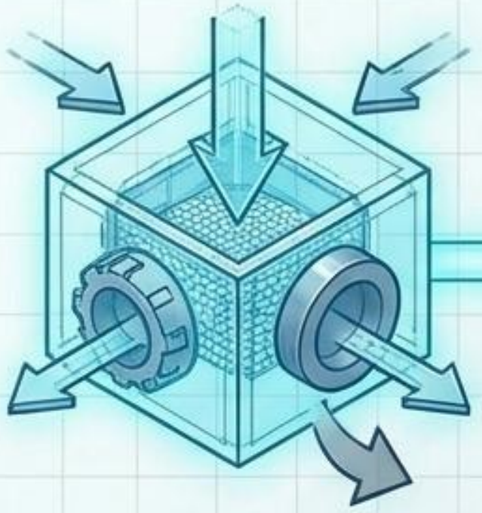
'protected' membocorkan data ke seluruh package!

1. **Paket Terbuka:** Di Java, SEMUA kelas dalam package yang sama memiliki akses ke atribut protected, bukan hanya subclass.
2. **Enkapsulasi Rusak:** Siapa pun dapat membuat subclass tak terbatas dan mengakses atribut secara langsung.

Best Practice: Selalu gunakan 'private' untuk atribut. Gunakan 'protected' hanya untuk metode internal.

Gemini Notebook

Arsitektur Data: Lebih Dari Sekadar Akses



Validasi Cerdas

Setter dapat menolak data buruk, melempar Exception, atau memformat ulang teks (misal: huruf kapital otomatis) sebelum disimpan ke dalam database.



Read-Only (Hanya Baca)

Dengan hanya menyediakan Getter (tanpa Setter), atribut menjadi Immutable dari luar kelas setelah objek dikonstruksi. Sangat aman untuk data kritis.



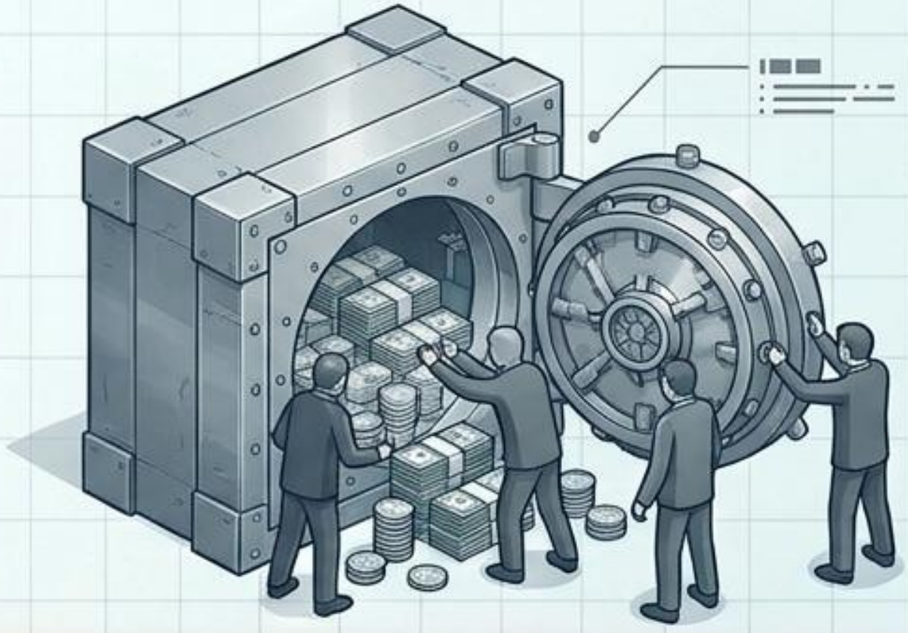
Write-Only (Hanya Tulis)

Menyediakan Setter tanpa Getter. Berguna untuk memasukkan password, PIN, atau security token yang tidak boleh dibaca ulang oleh sistem lain.

Gemini Notebook

Analogi Dunia Nyata: Mesin ATM

Tanpa Enkapsulasi - Bahaya



Akses Langsung

Nasabah mengambil uang langsung dari brankas. Rawan pencurian, salah hitung, dan sama sekali tidak ada catatan log transaksi.

Enkapsulasi - Aman

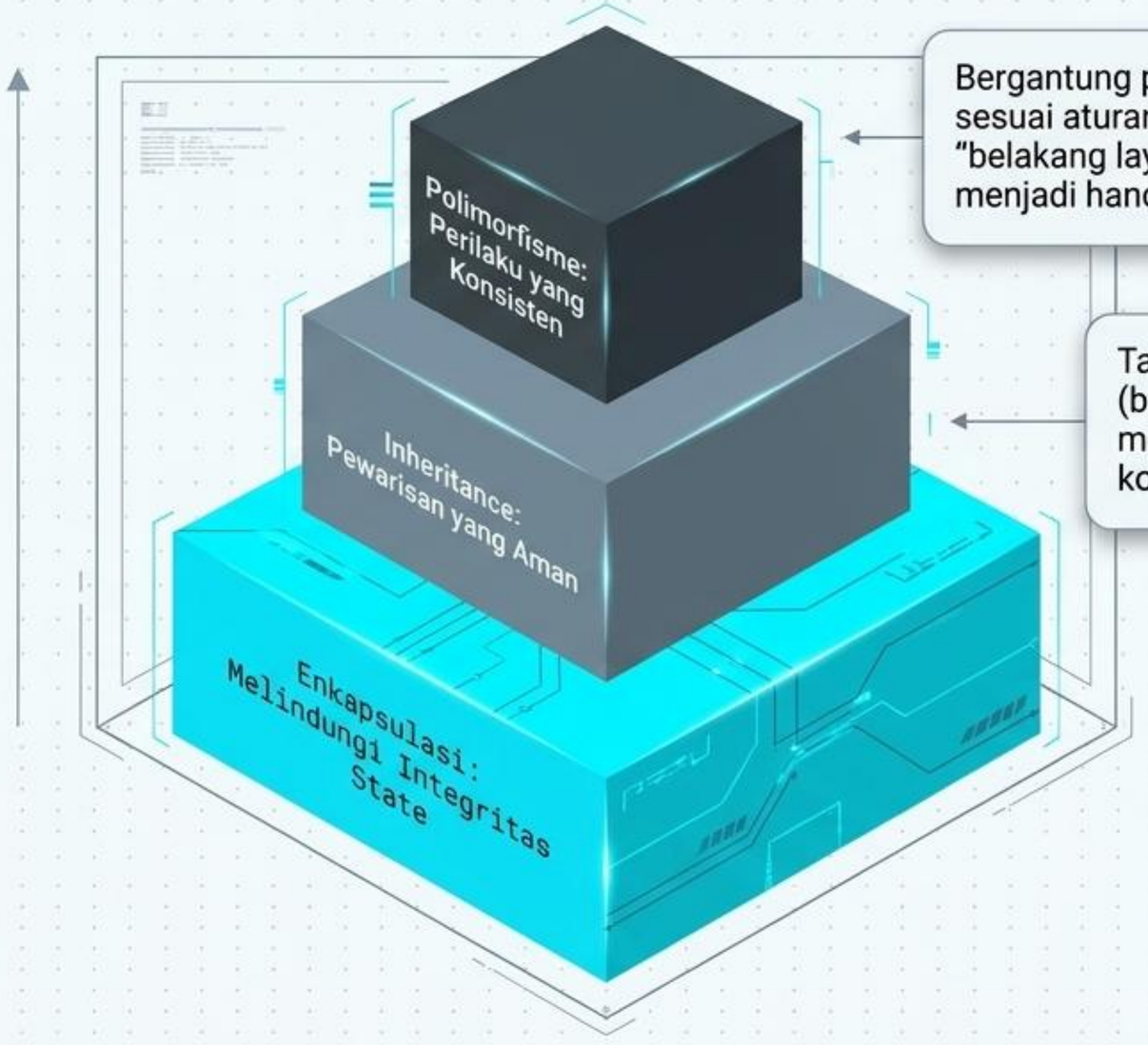


Antarmuka Publik

Nasabah berinteraksi dengan layar ATM. Saldo (private state) aman. Mesin memvalidasi PIN, memeriksa limit saldo, dan mencatat histori transaksi (public methods).

Gemini Notebook

Efek Domino Arsitektur Objek



Bergantung pada objek yang bertindak sesuai aturan. Jika data diubah dari "belakang layar", respons polimorfik menjadi hancur dan tidak terprediksi.

Tanpa Enkapsulasi, kelas dasar (base class) yang rapuh akan mewariskan state yang rusak atau korup ke semua subclass-nya.

Gemini Notebook

Ringkasan Eksekutif: 3 Aturan Emas

01

Default to Private

Jadikan ‘private’ sebagai default absolut untuk semua atribut kelas. Jangan buka akses kecuali benar-benar diperlukan oleh arsitektur.

02

Gunakan Gatekeeper

Bangun jalur komunikasi eksklusif menggunakan metode getter dan setter publik sebagai satu-satunya pintu masuk dan keluar data.

03

Lindungi State Objek

Jangan pernah percaya input eksternal. Gunakan metode setter untuk selalu memvalidasi logika bisnis (misal: cegah umur negatif) sebelum mengubah state.

Gemini Notebook